

Data Profiling Report

Food Inspection Datasets: Chicago and Dallas

Profiling scope: structure, row counts, null analysis, distinct categorical results, duplicate check, and rule-based anomaly identification. Where exact descriptive statistics were not visible in the provided outputs, the report documents only evidence that could be verified from the supplied materials.

Introduction

This report profiles two raw food inspection datasets used in the Food Inspections assignment: one from Chicago and one from Dallas. The goal of profiling is to understand source structure, data quality, and integration risks before standardizing the data into a common analytical model for reporting and dashboarding.

The two city datasets describe similar inspection activities but differ in schema and in how violations and scores are recorded. Chicago stores result categories and a single violations text field, while Dallas uses an inspection score and multiple violation memo fields. Because of these differences, profiling is essential before cleansing, standardization, and loading into the target dimensional model.

Overview

Basic Information

Metric	Profiled Value
Chicago raw row count	308,431
Dallas raw row count	78,984
Combined raw volume	387,415
Chicago visible source columns	17
Chicago source types observed	INTEGER, STRING, DATE, and location-related numeric fields
Dallas source characteristics observed	Includes inspection_score plus multiple violation memo columns
Gold/target model	Star schema with dim_date, dim_location, dim_restaurant, dim_inspection_type, dim_result, dim_violation, fact_inspection, and bridge_inspection_violation
Mapping coverage	54 mapped source-to-target rows in the supplied mapping workbook

Source and Target Data Types

Layer	Entity / Dataset	Key Fields	Observed / Expected Types
Source	bronze_chicago	inspection_id, license_num, zip, inspection_date, results, violations	INTEGER, INTEGER, INTEGER, DATE, STRING, STRING
Source	bronze_chicago	latitude, longitude, location	Numeric coordinate fields plus location text/structure
Source	bronze_dallas	inspection_score, violation_memo_-_1..3	INTEGER plus multiple STRING memo fields
Target	dim_date	date_key, full_date, month_num, year_num	INT, DATE, INT, INT
Target	dim_location	location_key, zip_code, latitude, longitude	BIGINT, STRING, DOUBLE, DOUBLE
Target	dim_restaurant	restaurant_key, business_name, license_number, is_current	BIGINT, STRING, STRING, BOOLEAN
Target	dim_result	result_key, standardized_result, pass_flag	BIGINT, STRING, BOOLEAN
Target	fact_inspection	inspection_key, inspection_score, violation_count	BIGINT, INT, INT

Data Quality Assessment

The profiling outputs show that core identifiers and inspection dates are mostly complete, but the source data still contains important quality issues that must be addressed in the silver layer before building reporting tables. The table below summarizes the main issues that could be verified from the supplied outputs.

Dataset	Column / Rule	Issue Type	Evidence	Impact	Recommended Action
Chicago	aka_name	Missing values	2,420 nulls (0.78%)	Alias information incomplete for some businesses	Keep nullable; do not invent aliases. Retain dba_name as primary business label.
Chicago	license_num	Missing values	19 nulls (0.01%)	Weakens business matching	Cast to string and preserve nulls; attempt business-level matching using name and address.
Chicago	facility_type	Missing values	5,323 nulls (1.73%)	Affects dashboard slicing by facility type	Standardize known values; assign UNKNOWN when business rule allows.
Chicago	risk	Missing values	88 nulls (0.03%)	Risk-category reporting gap	Trim/standardize and keep nullable if source is blank.
Chicago	city / state / zip	Missing values	city 189, state 67, zip 42	Location dimension quality risk	Backfill only from trusted location fields; invalid or missing zip should be rejected in silver when required.
Chicago	inspection_type	Missing values	1 null (<0.01%)	Minimal but affects type dimension lookup	Quarantine or map to UNKNOWN only with approval.
Chicago	violations	Missing values	86,503 nulls (28.05%)	Largest completeness	Parse available violation text,

				issue in the visible output	allow no-violation cases where valid, and store distinct inspection-violation rows.
Chicago	latitude / longitude / location	Missing values	1,027 / 1,037 / 1,039 nulls (~0.33%)	Reduces map completeness	Keep nullable for mapping; do not block fact load if standardized address and zip are present.
Chicago	full-row duplicates	Duplicate check	No rows returned from <code>groupBy(all columns).count().filter(count > 1)</code>	No evidence of exact duplicate records in the provided Chicago check	Retain duplicate rule in the pipeline and repeat for Dallas.
Chicago	results	Category standardization	7 distinct values observed	Cross-city reporting will be inconsistent without harmonization	Normalize to common reporting values in <code>dim_result</code> and derive Chicago score from result.
Dallas	inspection_score	Outlier / range issue	Distinct sample includes -26 alongside valid values such as 67, 70, 74, 93, 94	Negative score indicates invalid range and requires cleansing	Enforce valid numeric range, flag invalid records, and reject scores outside approved business rules.
Dallas	violation memo fields	Structural inconsistency	Violations are split across three memo columns rather than one field	Prevents direct union with Chicago schema	Stack memo fields into row-level violation records before building <code>dim_violation</code> and bridge table.

Summary Statistics

The supplied screenshots primarily show structural profiling rather than full descriptive statistics. The tables below therefore summarize the numerical and categorical evidence that could be directly verified from the provided outputs.

Numerical Features

Feature	Dataset	Profile Evidence	Interpretation
Row count	Chicago	308,431	Largest source; approximately 3.9 times the Dallas volume.
Row count	Dallas	78,984	Smaller source but still material for integration and reporting.
Combined raw volume	Both	387,415	Requires standardization and quality controls before dimensional loading.
Null count - violations	Chicago	86,503 (28.05%)	Largest visible completeness problem in the raw profile.
Null count - facility_type	Chicago	5,323 (1.73%)	Noticeable business classification gap.
Coordinate nulls	Chicago	latitude 1,027; longitude 1,037	Low relative rate, but relevant for map visuals.
Inspection score sample	Dallas	67, 70, 74, 93, 94, -26	The negative sample confirms range validation is required.

Categorical Features

Feature	Dataset	Observed Cardinality / Values	Comment
results	Chicago	7 values: Pass, No Entry, Not Ready, Out of Business, Business Not Located, Fail, Pass w/ Conditions	Needs normalization before cross-city comparison.
inspection_type	Chicago	Mostly complete; only 1 null observed	Appropriate as a small lookup dimension after standardization.
violations representation	Chicago	Single text field	Needs splitting into distinct violation rows.
violations representation	Dallas	Three memo columns	Needs stacking/unpivoting before integration.
source_city	Target standardization	2 expected values: Chicago, Dallas	Useful for lineage and cross-city filtering.

Data Transformation (if applicable)

Based on the profiling evidence and the supplied mapping workbook, the following transformations should be applied before loading the target model.

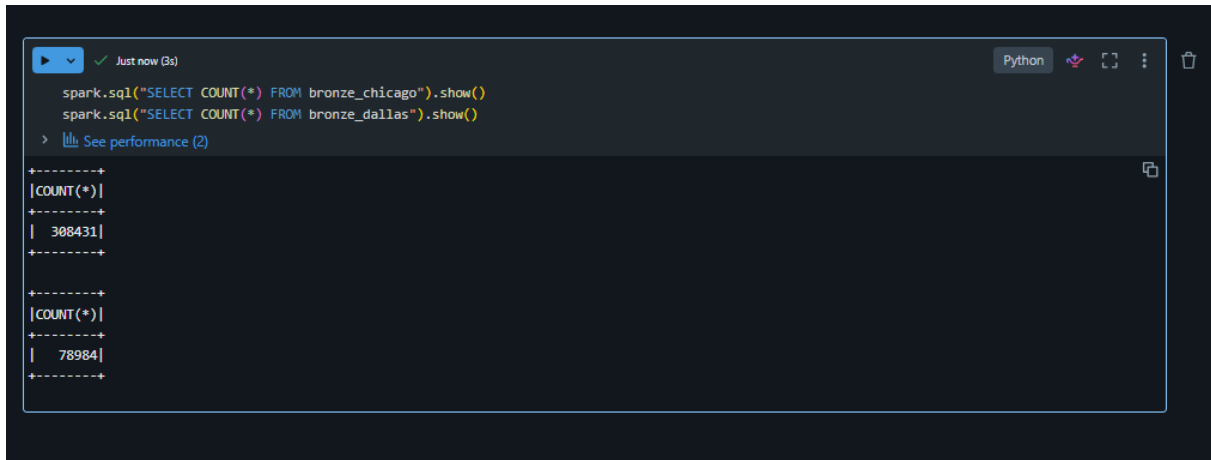
Transformation Area	Planned Transformation	Business Purpose
Column normalization	Convert column names to lowercase underscore format	Ensures consistent coding and easier schema handling across both sources.
Schema standardization	Rename and align Chicago and Dallas fields to a common silver schema	Supports unioning the datasets for downstream dimensional loads.
Date enrichment	Generate date_key and derive day, month, quarter, year, and weekday attributes	Supports time-based reporting in dim_date.
Type standardization	Cast zip_code to STRING, latitude/longitude to DOUBLE, and business identifiers to STRING where needed	Prevents type mismatch across sources.
Result harmonization	Standardize result categories and derive pass_flag for reporting	Creates a consistent dim_result across both cities.
Chicago score derivation	Map Chicago results to score values (Pass=90, Pass w/ Conditions=80, Fail=70, No Entry=0, else NULL)	Allows score-based reporting aligned with assignment rules.
Dallas score validation	Reject or quarantine invalid inspection_score values outside approved range	Prevents outlier scores from contaminating analytics.
Violation parsing	Split Chicago violations and stack Dallas violation memo columns into row-level records	Creates a consistent source for dim_violation and bridge_inspection_violation.
Deduplication	Retain distinct inspection-violation combinations only	Avoids duplicate violation counts in the fact layer.
Dimensional loading	Populate dim_date, dim_location, dim_restaurant, dim_inspection_type, dim_result, dim_violation, fact_inspection, and bridge_inspection_violation	Implements the target star schema for BI reporting.

Insights and Observations

- Chicago is the larger source by far, with 308,431 raw records versus 78,984 for Dallas.
- The most significant visible completeness issue in Chicago is the violations column, where 86,503 rows are null, followed by facility_type.
- Exact full-row duplicates were not found in the Chicago duplicate check shown in the provided outputs.
- Chicago result categories are heterogeneous and include operational statuses such as Not Ready and Business Not Located, so result standardization is necessary before cross-city reporting.
- Dallas inspection_score contains at least one negative sample value (-26), indicating that range validation must be enforced before loading fact data.
- The major cross-source integration challenge is structural, not volumetric: Chicago stores violations in one text field, while Dallas stores them across multiple memo columns.
- Geographic fields are mostly complete in Chicago, but low-level gaps in city, state, zip, and coordinates still matter for a clean location dimension and map-based dashboards.

Output Section

The following screenshots are the exact profiling outputs supplied for this assignment and are included here for documentation and submission support.



A screenshot of a Jupyter Notebook interface. The top bar shows a play button, a green checkmark, and the text 'Just now (3s)'. On the right, there is a 'Python' language selector and icons for sharing, full screen, and a trash can. The code cell contains two SQL queries: `spark.sql("SELECT COUNT(*) FROM bronze_chicago").show()` and `spark.sql("SELECT COUNT(*) FROM bronze_dallas").show()`. Below the code, there is a link '> See performance (2)'. The output shows two tables, each with a single row containing the count of rows. The first table is for 'bronze_chicago' with a count of 308431, and the second table is for 'bronze_dallas' with a count of 78984.

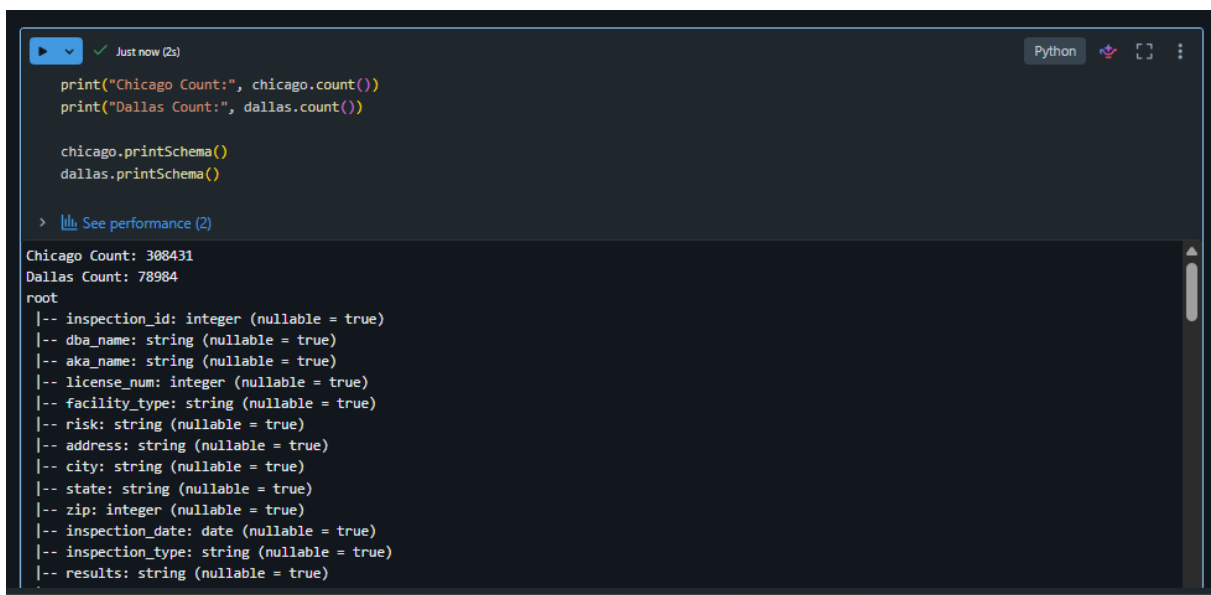
```
spark.sql("SELECT COUNT(*) FROM bronze_chicago").show()
spark.sql("SELECT COUNT(*) FROM bronze_dallas").show()

> See performance (2)
```

COUNT(*)
308431

COUNT(*)
78984

Figure 1. SQL validation of raw bronze row counts for Chicago and Dallas.



A screenshot of a Jupyter Notebook interface. The top bar shows a play button, a green checkmark, and the text 'Just now (2s)'. On the right, there is a 'Python' language selector and icons for sharing, full screen, and a trash can. The code cell contains two print statements for row counts: `print("Chicago Count:", chicago.count())` and `print("Dallas Count:", dallas.count())`, followed by `chicago.printSchema()` and `dallas.printSchema()`. Below the code, there is a link '> See performance (2)'. The output shows the row counts: 'Chicago Count: 308431' and 'Dallas Count: 78984', followed by the schema preview for the 'root' table, which lists various fields with their data types and nullability.

```
print("Chicago Count:", chicago.count())
print("Dallas Count:", dallas.count())

chicago.printSchema()
dallas.printSchema()

> See performance (2)
```

Chicago Count: 308431
Dallas Count: 78984

```
root
 |-- inspection_id: integer (nullable = true)
 |-- dba_name: string (nullable = true)
 |-- aka_name: string (nullable = true)
 |-- license_num: integer (nullable = true)
 |-- facility_type: string (nullable = true)
 |-- risk: string (nullable = true)
 |-- address: string (nullable = true)
 |-- city: string (nullable = true)
 |-- state: string (nullable = true)
 |-- zip: integer (nullable = true)
 |-- inspection_date: date (nullable = true)
 |-- inspection_type: string (nullable = true)
 |-- results: string (nullable = true)
```

Figure 2. DataFrame-based row counts and visible schema preview.

```
Just now (4s) Python
from pyspark.sql.functions import col, sum

def null_counts(df):
    return df.select([sum(col(c).isNull().cast("int")).alias(c) for c in df.columns])

null_counts(chicago).show()
null_counts(dallas).show()
> See performance \(2\)
```

inspection_id	dba_name	aka_name	license_num	facility_type	risk	address	city	state	zip	inspection_date	inspection_type	results	violations	latitude	longitude	location
7	1037	1039	2420	19	5323	88	0	189	67	42	0	1	0	86503	102	

Figure 3. Column-wise null-count output for the visible Chicago profile.

```
Just now (2s)
chicago.select("results").distinct().show()
dallas.select("inspection_score").distinct().show()
> See performance \(2\)
```

results
Pass
No Entry
Not Ready
Out of Business
Business Not Located
Fail
Pass w/ Conditions

inspection_score
70
93
67
-26
94
74

Figure 4. Distinct Chicago result categories and sample Dallas inspection_score values.

```
Just now (2s) Python
chicago.groupBy(chicago.columns).count().filter("count > 1").show()
> See performance (1)
```

inspection_id	dba_name	aka_name	license_num	facility_type	risk	address	city	state	zip	inspection_date	inspection_type	results	violations	latitude	longitude	location	count

Figure 5. Exact full-row duplicate check for Chicago.

Conclusion and Recommendations

The profiling evidence shows that the food inspection sources are suitable for integration, but they require schema harmonization and targeted cleansing before analytical use. Core identifiers and inspection dates are generally reliable, while the main data-quality issues are missing violations in Chicago, missing business classification fields, and invalid Dallas score values.

Before moving fully into the silver and gold layers, the recommended next steps are to enforce required-field checks, validate zip and score ranges, standardize result values, parse violations into row-level records, and repeat duplicate and null profiling after cleansing. A follow-up notebook run should also generate full descriptive statistics for Dallas inspection_score and additional cardinality summaries so the profiling package is even stronger for final submission.